The same technique works for other sources of events. Using the `withEvents` function, you can map and filter user events, such as keyboard interaction.

9.4.4. Client side

It’s time to look at that game of Pong again, because the same technique that worked for a data stream works for user input data. The paddles in a game of Pong are moved up and down the screen based on keyboard presses performed by the players. Suppose the game of Pong will run in a browser and that the HTML structure is as follows:

```
<!DOCTYPE html>
<html dir="ltr" lang="en-US">
  <head>
    <meta http-equiv="Content-Type" content="text/html;
charset=utf-8">
    <title>Pong</title>
  </head>
  <body>
    <div id="pong"></div>
  </body>
  <script src="9.5.js"></script>
</html>
```

Attaching a handler to keyboard events means that the data supplied by the keyboard (the key presses) is received:

```
document.on 'keypress', (event) ->
  console.log 'The keyboard was pressed'
```

Sure, you could write an entire program inside the keypress handler, but that will eventually lead to event handler spaghetti. In the next listing you see the paddle-controlling code for Pong implemented by treating the key presses as a stream of event data. Compile this listing and use it with the sample HTML shown previously to experiment.

Listing 9.5. Controlling paddles in a web browser